Generative Adversarial Networks:

Recent Advances and Popular Application

Yongcheng Jing

College of Computer Science and Technology

Zhejiang University

Mar. 20th, 2017

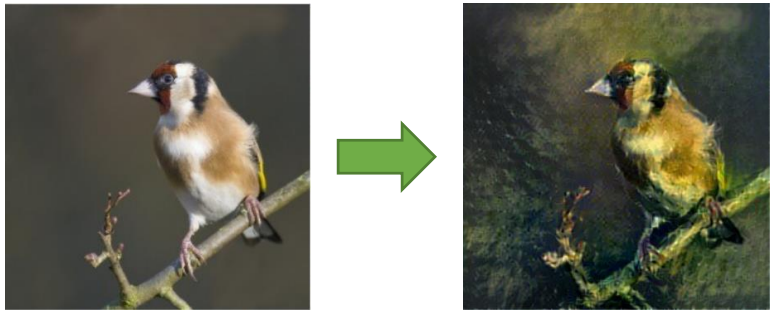
Review of the Original GAN

- GAN is an example of **Generative Model**.
  - **Generative Model** refers to any model that **takes a training set**, consisting of samples drawn from a distribution p_{data} , and learns to **represent an estimate p_{model} of that distribution**.

Review of the Original GAN

- GAN is an example of **Generative Model**.
  - **Generative Model** refers to any model that **takes a training set**, consisting of samples drawn from a distribution p_{data} , and learns to **represent an estimate p_{model} of that distribution**.
  - Examples of Generative Model applications:

Image Super-resolution



"Fast" Neural Style Transfer

Sketches to Images

Review of the Original GAN

- Deep Generative Models **prior to GAN**[1]:
  - Boltzmann machine, Variational autoencoder, GSN, Nonlinear ICA, etc.
- **Advantages** of GAN over these prior models:
  - The design of **generator function** has very **few restrictions**.
  - **No Markov chains** are needed. (Markov chain methods have the drawbacks of slow convergence, no clear way to test whether the chain has converged, etc.)
  - Often regarded as **producing the best samples**.
  - … …

1. Goodfellow, I. (2016). NIPS 2016 Tutorial: Generative Adversarial Networks. arXiv preprint arXiv:1701.00160.

Review of the Original GAN

- Basic idea of GAN is to set up a **game** between two players, **generator** **vs** **discriminator**.
  - The generator **creates samples** that are intended to come from the **same distribution** as the **training data**.
  - The discriminator examines samples to **determine** they are **real** or **fake**.
  - The generator is trained to **fool** the discriminator until the generated data is **indistinguishable**.

Review of the Original GAN

- How to use mathematics to model **generator VS discriminator** ?
  - The **Binary Cross-Entropy (BCE) cost function** is a good choice[2].
    - $\mathrm{BCE} = -\frac{1}{n}\sum_x [y ln a + (1-y)\ln(1-a)]$

$\boldsymbol{x}$: training sample.
$\boldsymbol{n}$: # of x.
$\mathbf{y}$: label {0,1}.
$\boldsymbol{a}$: output of network.
***Objective*:** $(y = 0 \& a \approx 0) \& (y = 1 \& a \approx 1)$

2. **BCE's Derivative is beautiful:** http://neuralnetworksanddeeplearning.com/chap3.html#the_cross-entropy_cost_function

Review of the Original GAN

- How to use mathematics to model **generator VS discriminator** ?
  - The **Binary Cross-Entropy (BCE) cost function** is a good choice[2].
    - $\mathrm{BCE} = -\frac{1}{n}\sum_x [y \ln a + (1-y)\ln(1-a)]$

x: training sample.
n: # of x.
y: label {0,1}.
a: output of network.
Objective: $(y = 0 \& a \approx 0) \& (y = 1 \& a \approx 1)$

  - For **BCE** in **GAN:**
    - Discriminator's cost

$$J^{(D)}\left(\theta^{(D)}, \theta^{(G)}\right) = -E_{x\sim p_{data}}[\log D(x)] - E_{z\sim p_z}[\log(1 - D(G(z)))]$$

    - Generator's cost (zero-sum games)

$$J^{(G)} = -J^{(D)}\left(\theta^{(D)}, \theta^{(G)}\right)$$

$x \sim p_{data}$: x follows the distribution of training data.
$z \sim p_z$: random noise
z follows the distribution of some simple prior distribution, $e.g.$ Gaussian

2. **BCE's Derivative is beautiful:** http://neuralnetworksanddeeplearning.com/chap3.html#the_cross-entropy_cost_function

Review of the Original GAN

- Discriminator's cost

$$J^{(D)}\left(\theta^{(D)}, \theta^{(G)}\right) = -E_{x\sim p_{data}}[\log D(x)] - E_{z\sim p_z}[\log(1 - D(G(z)))]$$

$\boldsymbol{D(m) = 1}$ if the discriminator thinks that m comes from real samples.
$\boldsymbol{D(m) = 0}$ if m comes from the generator.
$\boldsymbol{J^{(D)} \geq 0}$

- Generator's cost

$$J^{(G)} = -J^{(D)}\left(\theta^{(D)}, \theta^{(G)}\right)$$

- Zero-sum games are also minimax games:

$$\text{Objective} = min_G max_D (E_{x\sim p_{data}}[\log D(x)] + E_{z\sim p_z}\left[\log(1 - D\left(G(z)\right))\right])$$

Review of the Original GAN

- Discriminator's cost

$$J^{(D)}\left(\theta^{(D)},\theta^{(G)}\right) = -E_{x\sim p_{data}}[\log D(x)] - E_{z\sim p_z}[\log(1-D(G(z)))]$$

$\boldsymbol{D(m) = 1}$ if the discriminator thinks that m comes from real samples.
$\boldsymbol{D(m) = 0}$ if m comes from the generator.
$\boldsymbol{J^{(D)} \geq 0}$

- Generator's cost

$$J^{(G)} = -J^{(D)}\left(\theta^{(D)},\theta^{(G)}\right)$$

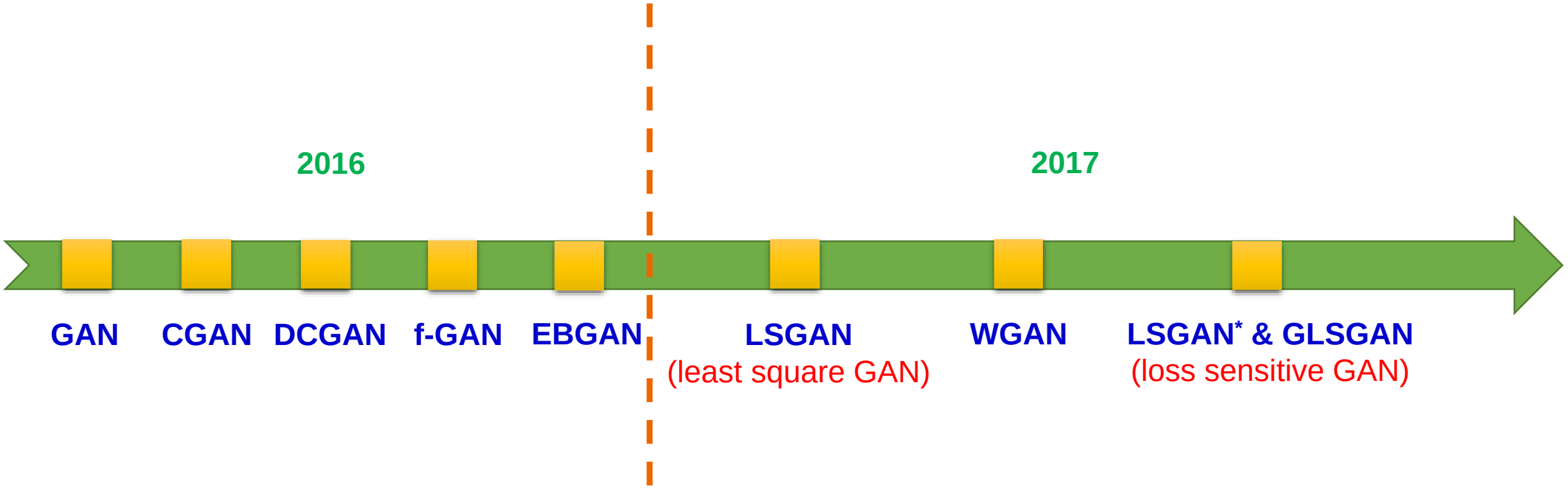
- Zero-sum games are also minimax games:

$$\text{Objective} = min_G max_D (E_{x\sim p_{data}}[\log D(x)] + E_{z\sim p_z}\left[\log(1-D\left(G(z)\right))\right])$$

- **Other available generator cost** function except for zero-sum games?
  - Heuristic, non-saturating game
  - Maximum likelihood game
- See Section 3.2 in 1 for more details and comparisons of three variations.

1. Goodfellow, I. (2016). NIPS 2016 Tutorial: Generative Adversarial Networks. arXiv preprint arXiv:1701.00160.

Development in GAN Theory



3. https://github.com/zhangqianhui/AdversarialNetsPapers

Development in GAN Theory

- For GAN, CGAN and DCGAN, refer to 4
- fGAN: Training Generative Neural Samplers using Variational Divergence Minimization
- EBGAN: Energy-based Generative Adversarial Network (LeCun's paper)
- LSGAN[5]: Least Squares Generative Adversarial Networks
- WGAN[6]:
  - (1) Towards Principled Methods for Training Generative Adversarial Networks
  - (2) Wasserstein GAN
- LSGAN[7] & GLSGAN[8]
  - Loss-Sensitive Generative Adversarial Networks on Lipschitz Densities

4. Jie Lei (2016.11.7).Seminar about Generative Adversarial Nets in VIPA.
5. LSGAN: https://zhuanlan.zhihu.com/p/25768099?utm_source=qq&utm_medium=social
6. WGAN: https://zhuanlan.zhihu.com/p/25071913
7. LSGAN: https://zhuanlan.zhihu.com/p/25204020?group_id=818602658100305920
8. GLSGAN: https://zhuanlan.zhihu.com/p/25580027

CVPR 2017

Image-to-Image Translation with Conditional Adversarial Networks

Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, Alexei A. Efros

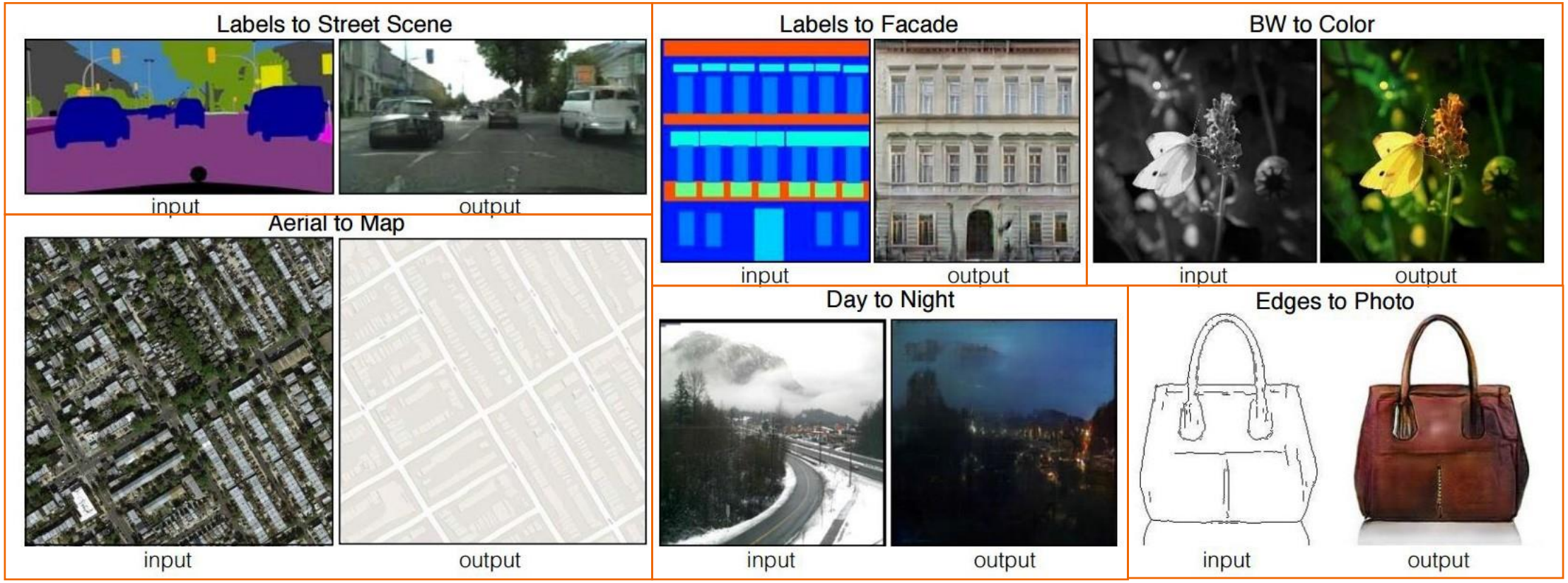
Berkeley AI Research (BAIR) Laboratory

Outline

- Introduction
- Proposed Method
- Experiment
- Conclusions

Introduction: Image to Image

- What is **Image to Image Translation?**
  - Translating one possible representations of a scene into another.



Introduction: Previous Work

- There are already lots of state-of-the-art researches for each image-to-image translation problem.
  - **Edges2Photo:** Sketch2photo: internet image montage. (TOG)
  - **Day2Night:** Data-driven hallucination of different times of day from a single outdoor photo (TOG)
  - **BW2Color:** Colorful image colorization (ECCV)

Introduction: Previous Work

- There are already lots of state-of-the-art researches for each image-to-image translation problem.
  - **Edges2Photo:** Sketch2photo: internet image montage. (TOG)
  - **Day2Night:** Data-driven hallucination of different times of day from a single outdoor photo (TOG)
  - **BW2Color:** Colorful image colorization (ECCV)
- But each of these tasks are tackled with **separate specific-purpose machinery**.

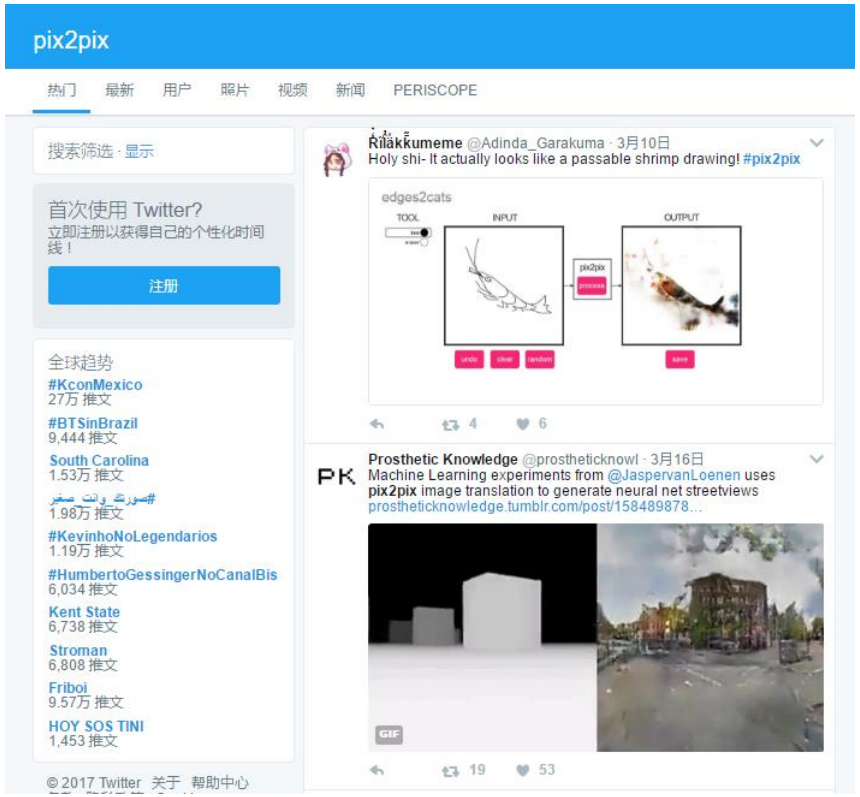
Introduction: Motivation

- This paper aims to develop a common framework for **all these problems**.
- Contributions:
  - #1. Demonstrate that on a wide variety of problems, **conditional GANs produce reasonable results**.
  - #2. On a variety of problems, present **a simple framework** sufficient to achieve good results and to analyze the effects of several import choices.
- Code:
  - https://github.com/phillipi/pix2pix (Torch)
  - https://github.com/yenchenlin/pix2pix-tensorflow (TensorFlow r0.11, Cuda8 needed)
  - https://github.com/affinelayer/pix2pix-tensorflow (TensorFlow 1.0.0, Cuda8 needed)

(See Appendix for installation instruction)

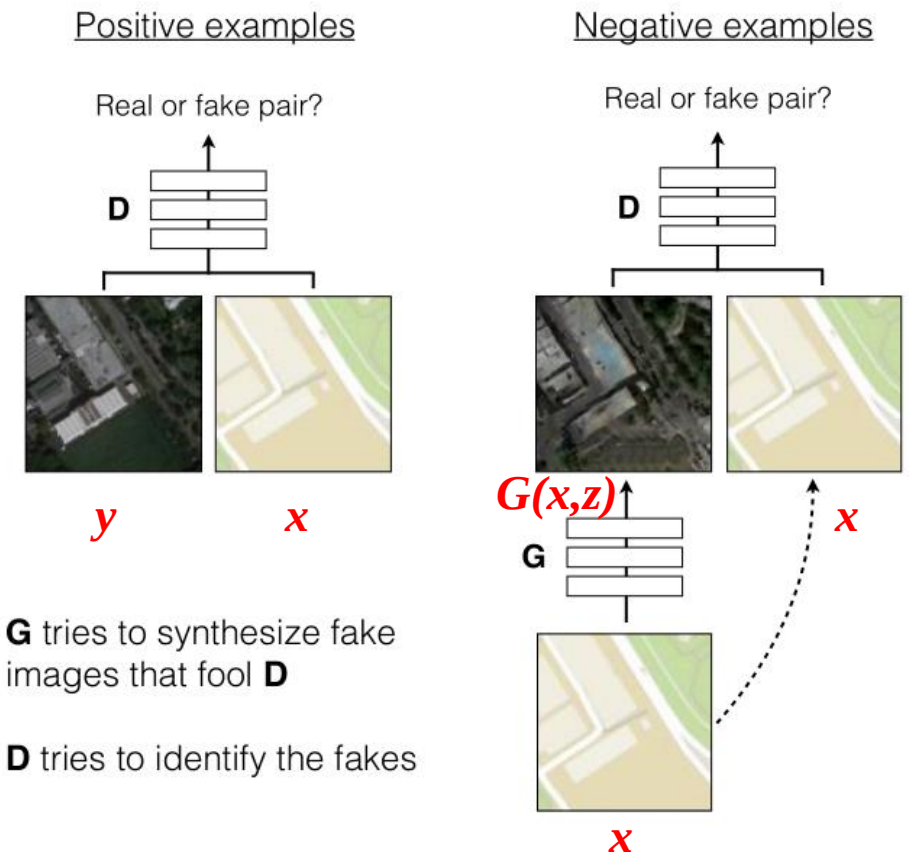
Introduction: Industrial Application

- Industrial applications:
  - **web app:** https://affinelayer.com/pix2pix/
  - **ios app:** doodle.ai
- Popular in Twitter:



Proposed Method: Objective

- cGAN Loss:
  - $L_{cGAN}(G,D) = E_{\boldsymbol{x\sim p_{data_1}, y\sim p_{data_2}}}[\log D(x,y)] - E_{\boldsymbol{x\sim p_{data_1}}, z\sim p_z(z)}[\log(1 - D(x, G(x,z))]$
  - e.g.



Proposed Method: Objective

- cGAN Loss:
  - $L_{cGAN}(G, D) = E_{\boldsymbol{x} \sim \boldsymbol{p}_{\boldsymbol{data_1}}, \boldsymbol{y} \sim \boldsymbol{p}_{\boldsymbol{data_2}}}[\log D(x, y)] - E_{\boldsymbol{x} \sim \boldsymbol{p}_{\boldsymbol{data_1}}, \boldsymbol{z} \sim \boldsymbol{p}_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(x, G(x, z))]$
  - To produce **stochastic output**, if we just **add random noise $\boldsymbol{z}$** to $\boldsymbol{x}$ as previous cGANs do:

Proposed Method: Objective

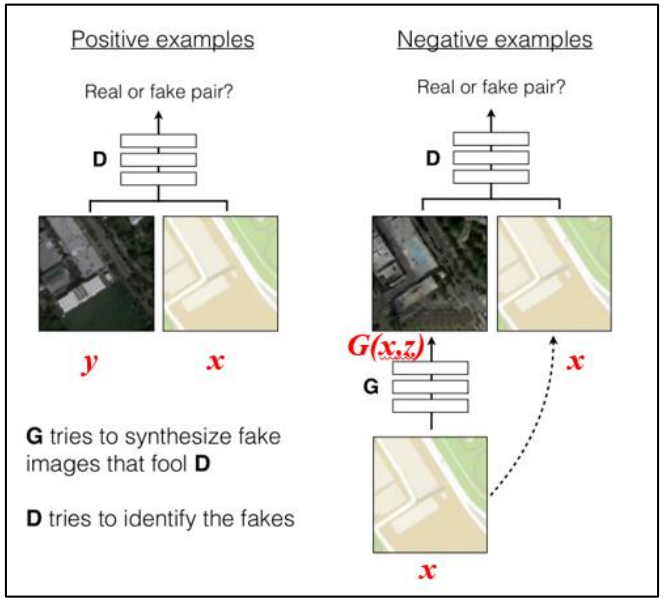
- cGAN Loss:
  - $L_{cGAN}(G,D) = E_{\boldsymbol{x}\sim \boldsymbol{p}_{\boldsymbol{data_1}},\boldsymbol{y}\sim \boldsymbol{p}_{\boldsymbol{data_2}}}[\log D(x,y)] - E_{\boldsymbol{x}\sim \boldsymbol{p}_{\boldsymbol{data_1}},\boldsymbol{z}\sim \boldsymbol{p}_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(x, G(x,z))]$
  - To produce **stochastic output**, if we just **add random noise $\boldsymbol{z}$** to $\boldsymbol{x}$ as previous cGANs do:
    - The generator just simply learned to **IGNORE THE NOISE!**

Proposed Method: Objective

- cGAN Loss:
  - $L_{cGAN}(G,D) = E_{\boldsymbol{x}\sim \boldsymbol{p}_{\boldsymbol{data_1}},\boldsymbol{y}\sim \boldsymbol{p}_{\boldsymbol{data_2}}}[\log D(x,y)] - E_{\boldsymbol{x}\sim \boldsymbol{p}_{\boldsymbol{data_1}},\boldsymbol{z}\sim \boldsymbol{p}_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(x, G(x,z))]$
  - To produce **stochastic output**, if we just **add random noise $\boldsymbol{z}$** to $\boldsymbol{x}$ as previous cGANs do:
    - The generator just simply learned to **IGNORE THE NOISE!**
  - Thus, provide noise in the form of **dropout**.

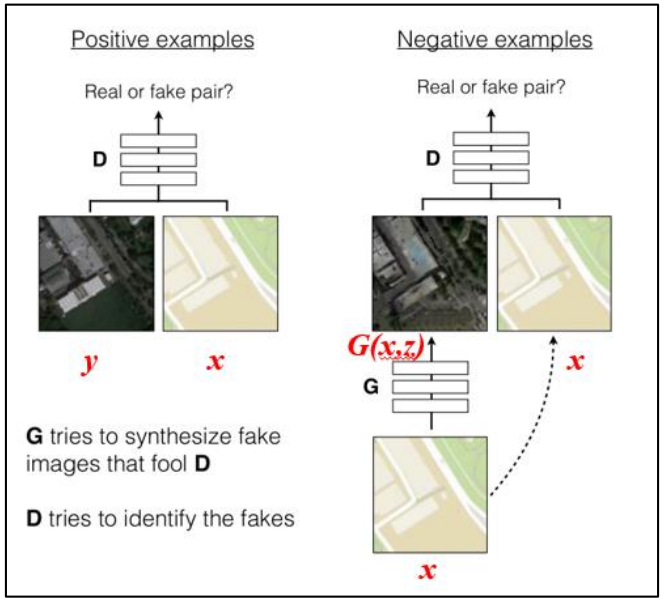
Proposed Method: Objective

- cGAN Loss:
  - $L_{cGAN}(G,D) = E_{x\sim p_{data_1}, y\sim p_{data_2}}[\log D(x,y)] - E_{x\sim p_{data_1}, z\sim p_z(z)}[\log(1 - D(x, G(x,z))]$
- L1 Loss (to encourage the generated image to be **near the GT output**)
  - $L_{L1}(G) = E_{x\sim p_{data_1}, y\sim p_{data_2}, z\sim p_z(z)}[\|y - G(x,z)\|_1]$
  - L2 distance produces more blur.



Proposed Method: Objective

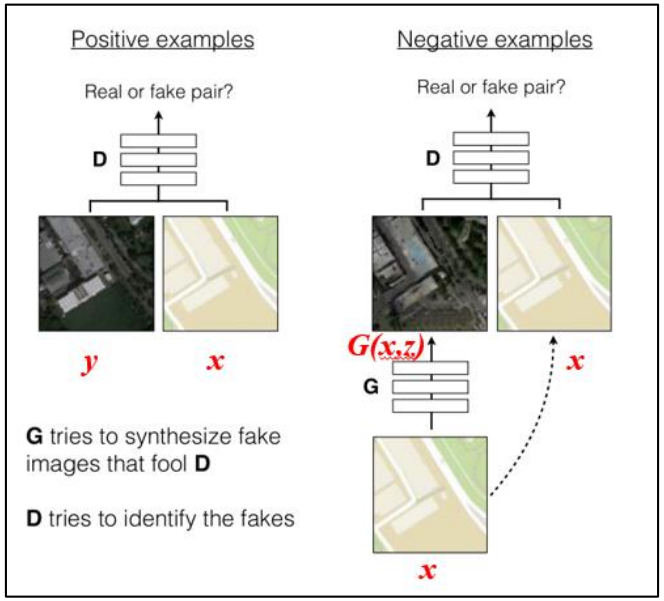
- cGAN Loss:
  - $L_{cGAN}(G,D) = E_{x\sim p_{data_1}, y\sim p_{data_2}}[\log D(x,y)] - E_{x\sim p_{data_1}, z\sim p_z(z)}[\log(1 - D(x, G(x,z))]$
- L1 Loss (to encourage the generated image to be **near the GT output**)
  - $L_{L1}(G) = E_{x\sim p_{data_1}, y\sim p_{data_2}, z\sim p_z(z)}[\|y - G(x,z)\|_1]$
  - L2 distance produces more blur.
- **Final Objective:**
  - $G^* = \arg min_G max_D (L_{cGAN}(G,D) + \lambda L_{L1}(G))$



Proposed Method: Objective

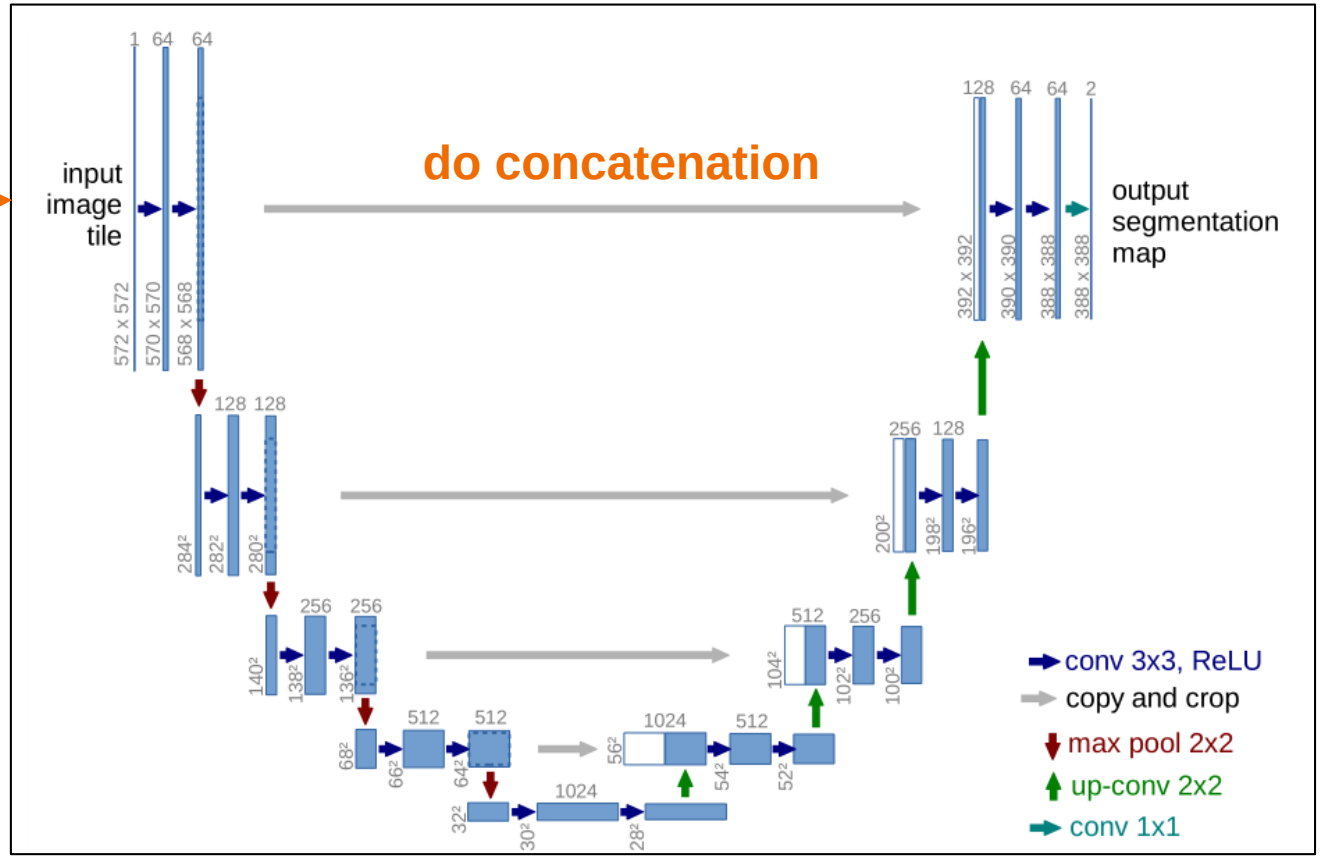
- cGAN Loss:
  - $L_{cGAN}(G,D) = E_{x\sim p_{data_1}, y\sim p_{data_2}}[\log D(x,y)] - E_{x\sim p_{data_1}, z\sim p_z(z)}[\log(1 - D(x, G(x,z))]$
- L1 Loss (to encourage the generated image to be **near the GT output**)
  - $L_{L1}(G) = E_{x\sim p_{data_1}, y\sim p_{data_2}, z\sim p_z(z)}[\|y - G(x,z)\|_1]$
  - L2 distance produces more blur.
- **Final Objective:**
  - $G^* = \arg min_G max_D (L_{cGAN}(G,D) + \lambda L_{L1}(G))$

? ?



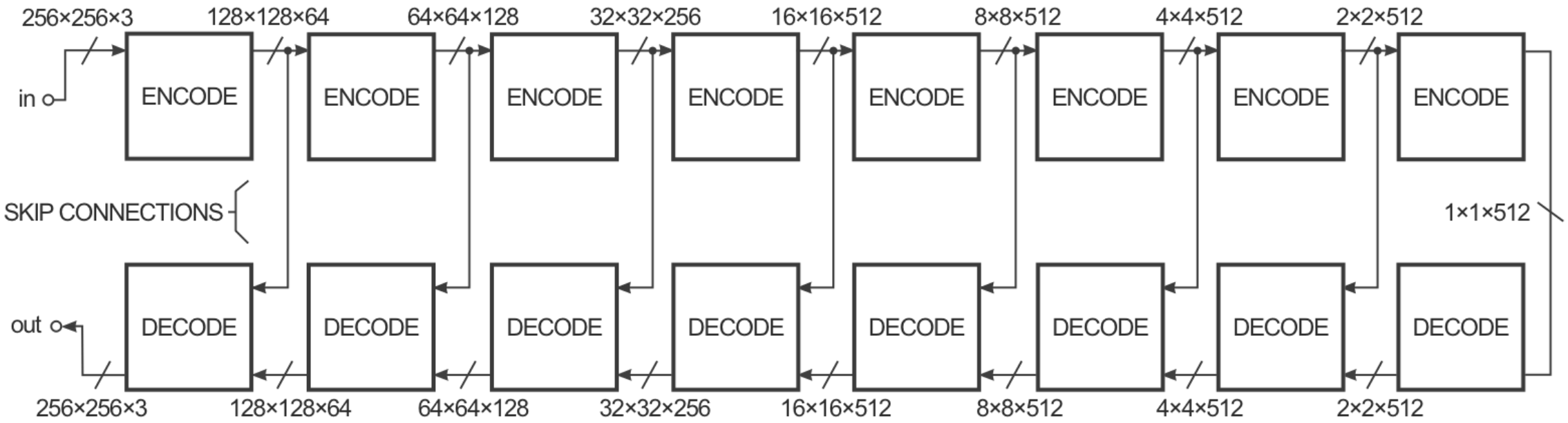
Proposed Method: Network Architecture

- Generator
  - Intuition: Lots of low-level information **shared** between the input and output.
  - Based on U-Net with **skips**.
    - What is skips?
    - e.g. U-Net



Proposed Method: Network Architecture

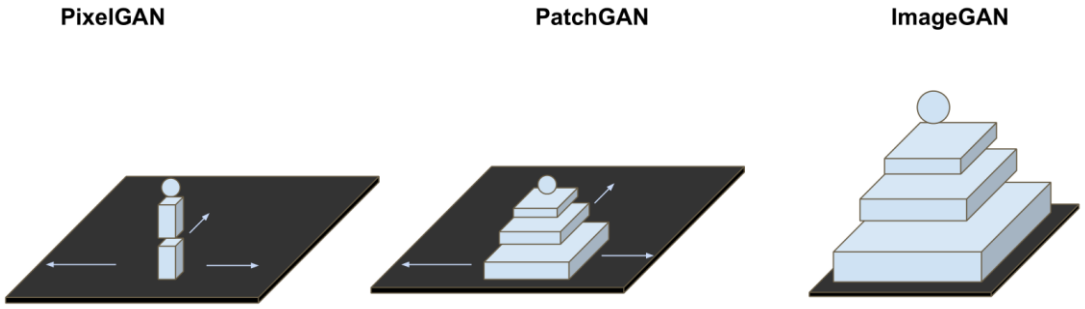
- Generator
  - Final Architecture of generator:



with dropout

Proposed Method: Network Architecture

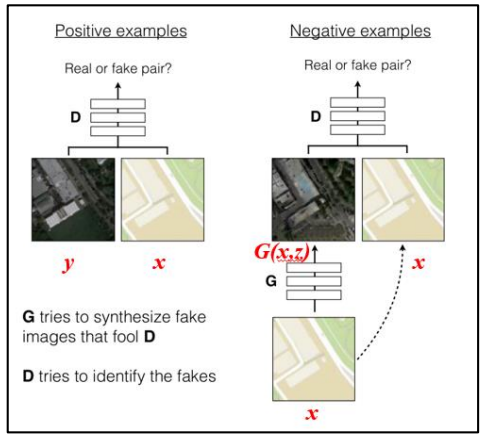
- Discriminator
  - Intuition: Divide image into patches can reduce parameters, run faster and be applied on arbitrarily large images.
  - Based on **PatchGAN**[9]
    - Try to classify if **each $N \times N$ patch** in an image is real or fake.
    - **Averaging** all responses to provide the ultimate output of D.

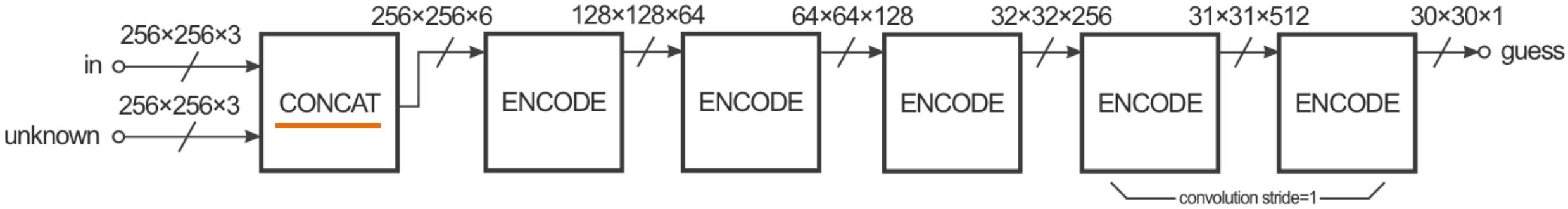


9. Li, Chuan, and Michael Wand. "Precomputed real-time texture synthesis with markovian generative adversarial networks." European Conference on Computer Vision. Springer International Publishing, 2016.

Proposed Method: Network Architecture

- Discriminator
  - Final Architecture of discriminator:





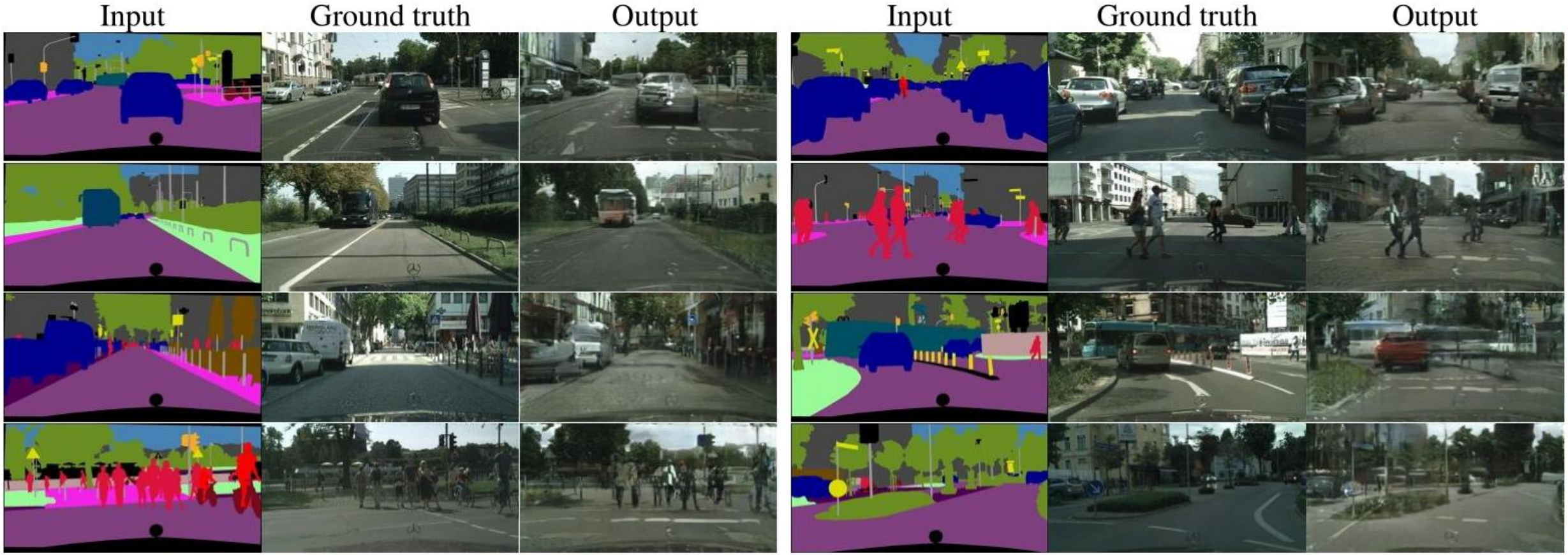
Proposed Method: Optimization

- Minibatch SGD
- Adam
- **Instance normalization**[10] (or contrast normalization)
  - Batch normalization with **batch size equals to 1**
  - Good for Neural Style Transfer as the contrast of content image should be discarded and it can also make the training objective easier to learn.
  - For the problem in this paper, **little difference** between Batch Normalization and Instance Normalization.

10. Ulyanov, D., Vedaldi, A., & Lempitsky, V. (2016). Instance Normalization: The Missing Ingredient for Fast Stylization. arXiv preprint arXiv:1607.08022.

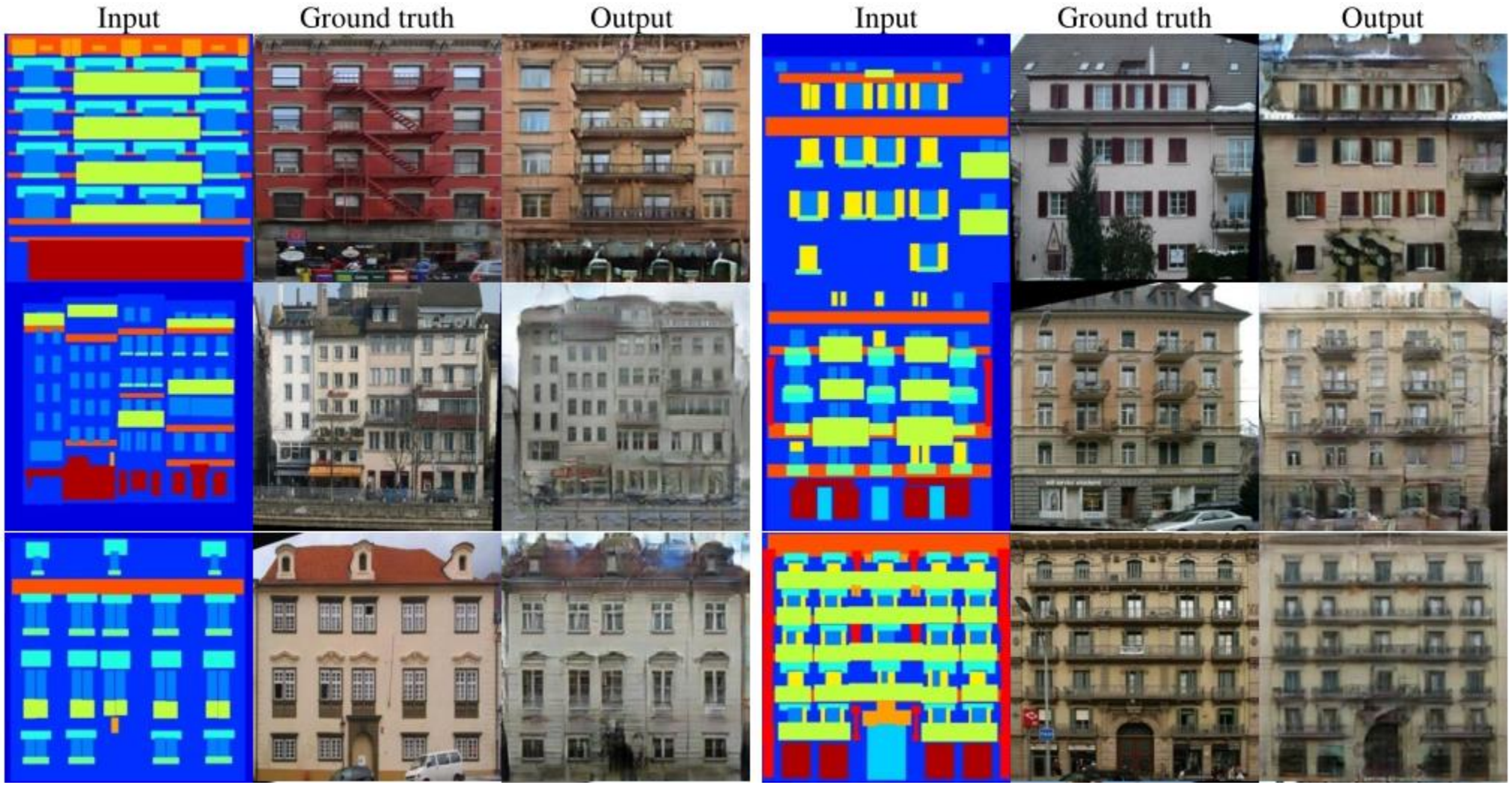
Experiment: Qualitative Evaluation

- Labels to Images



Experiment: Qualitative Evaluation

- Labels to Images

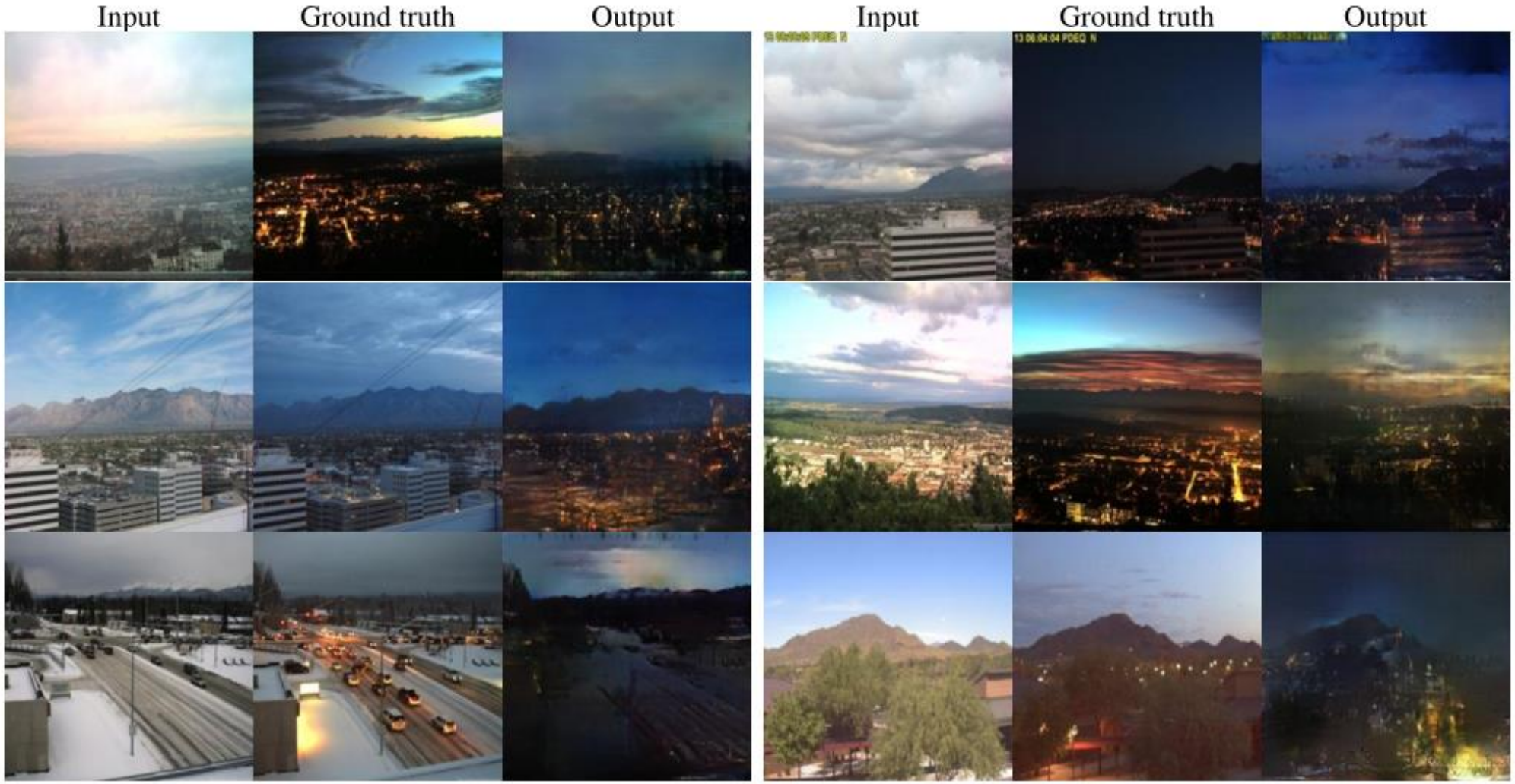


Experiment: Qualitative Evaluation

- Sketches to Images

Experiment: Qualitative Evaluation

- Day to Night



Experiment: Quantitative Evaluation

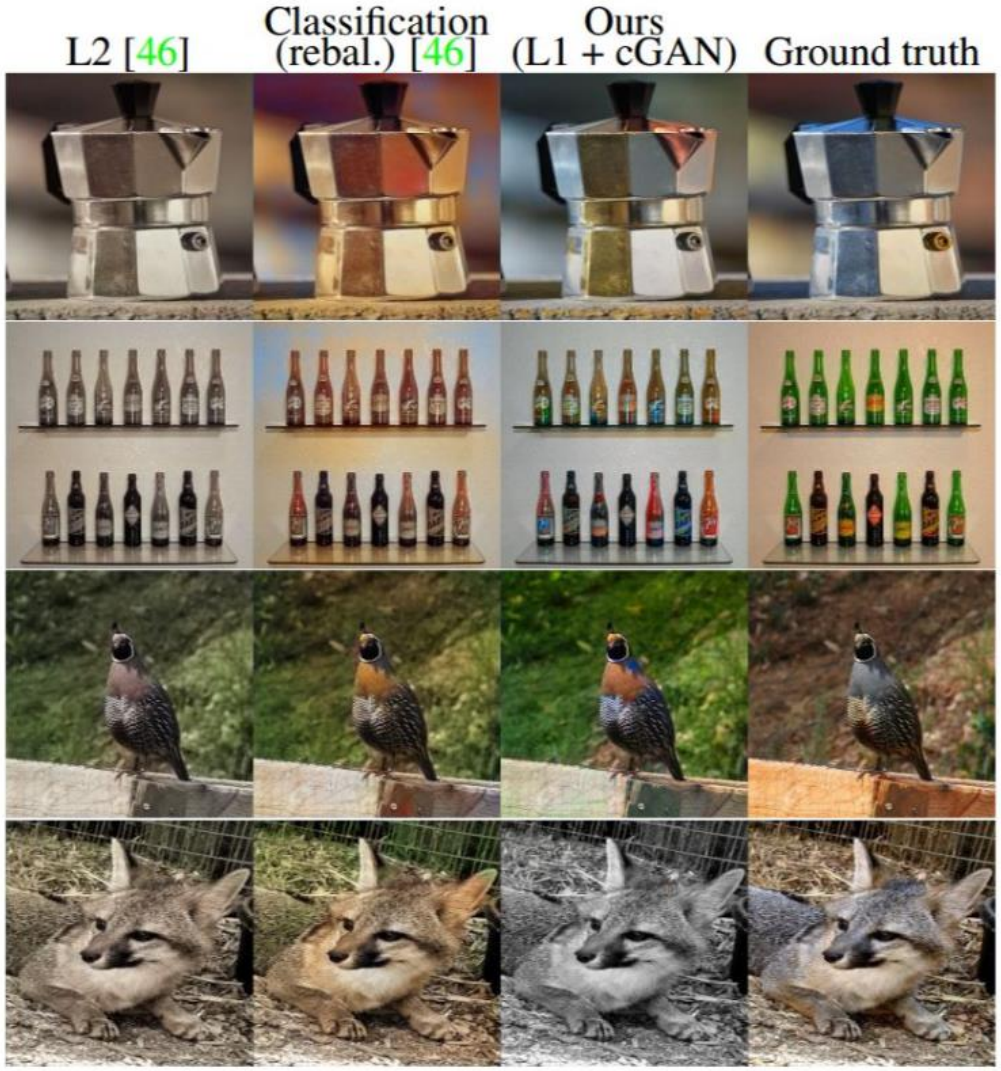
- Evaluation criterion:
  - AMT perceptual studies (Human Discriminator)
  - FCN-score
    - Intuition: If the results are realistic, semantic segmentation method FCN can be able to **segment the objects** in the result image.
    - Use the accuracy of semantic segmentation to compare the results.

Experiment: Quantitative Evaluation

- BW to Color

	L2	Classification (rebal.)	L1+cGAN
Labeled as real	16.3%	**27.8%**	22.5%

ECCV 2016 Paper



Experiment: Quantitative Evaluation

- Labels to Images

$$\frac{True \cap Predicted}{True \cup Predicted}$$

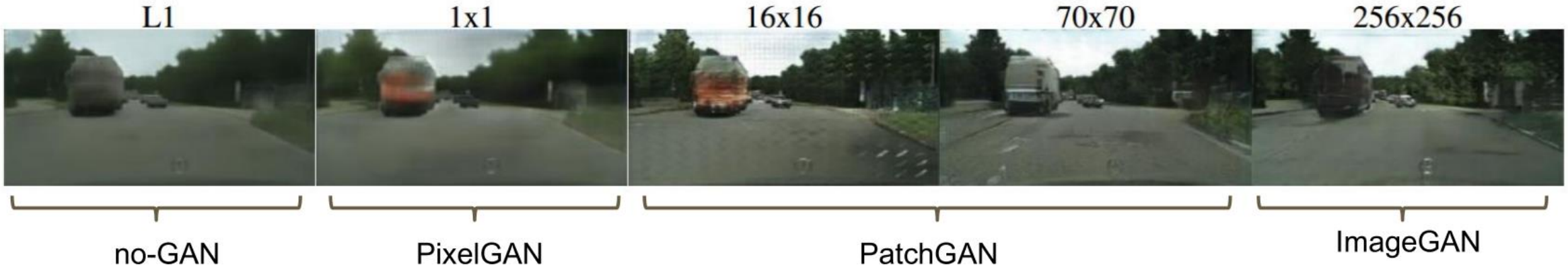
- FCN-score

Loss	Per-pixel acc.	Per-class acc.	Class IOU
L1	0.44	0.14	0.10
GAN	0.22	0.05	0.01
cGAN	0.61	**0.21**	**0.16**
L1+GAN	**0.64**	0.19	0.15
L1+cGAN	0.63	**0.21**	**0.16**
Ground truth	0.80	0.26	0.21

Experiment: Quantitative Evaluation

- Labels to Images

Discriminator receptive field	Per-pixel acc.	Per-class acc.	Class IOU
1×1	0.44	0.14	0.10
16×16	0.62	0.20	**0.16**
70×70	**0.63**	**0.21**	**0.16**
256×256	0.47	0.18	0.13



Conclusions

- **Conditional Adversarial Networks** are a promising approach for many image to image translation tasks.
- Using **U-net** as a generator has been a big improvement for **forwarding low level features** through the network and partially reconstructing it at the output.
- Using the **PatchGAN** approach we can train and generate high resolution images.

Thanks!